Hash tables

Part III Data Structures

Scope

Hash tables map keys to save space in a sparse universe, but speed depends on collisions and load. Separate arguments establish the costs of chaining, universal families, probe permutations, and two-level perfect hashing. Tombstones, duplicate prevention, and unsuccessful-query traces make the conditions behind expected and worst-case guarantees explicit.

Prerequisites

Probability, modular arithmetic, and average-case analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Derive expected hashing costs from direct addressing.
- Evaluate the assumptions of division, multiplication, and universal hashing.
- Analyze open-address probing and perfect hashing.
- 11.1 Direct-address tables
- 11.2 Hash tables
- 11.3 Hash functions
- 11.4 Open addressing
- 11.5 Perfect hashing

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Direct-address tables

A dictionary maintains a key→record mapping. Here each key has at most one record, and inserting an existing key updates its record. If the universe $U=\{0,\ldots,M-1\}$ is small enough, use $T[key]$ directly. EMPTY marks an unused slot and is distinct from any legal record.

Search reads $T[k]$, insert writes $T[k]$, and delete clears it. Each takes worst-case $O(1)$, with $\Theta(M)$ space. Initializing all slots to EMPTY usually takes $\Theta(M)$; constant query time does not imply constant construction time. A bit set uses one bit per key but stores no satellite data.

For $U=0..9$ with keys 2,5,8, only $T[2]$, $T[5]$, and $T[8]$ point to records. The key's numerical value supplies the index, with no collisions. Reading and converting a long string key costs time proportional to its length. Subsequent $O(1)$ hashing and equality claims assume keys handled in a fixed number of word operations.

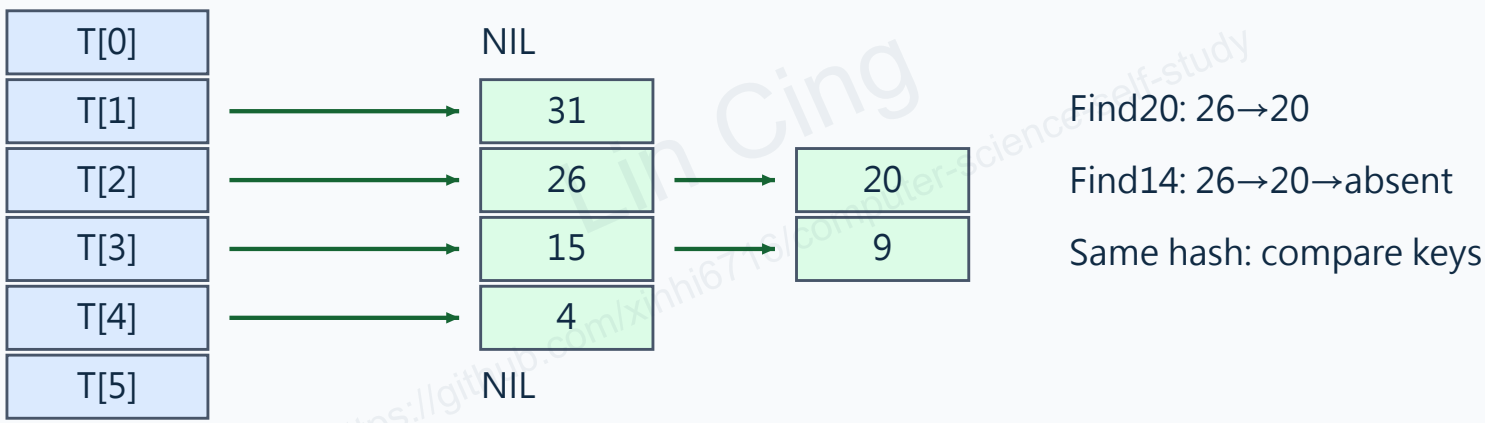
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chaining

When $|U|$ greatly exceeds the n stored records, use m slots and $h:U\to\{0,\ldots,m-1\}$. If $|U|>m$, the pigeonhole principle guarantees a collision between distinct keys for every fixed h. Chaining links records in the same slot. Search must compare original keys: equal hashes do not imply equal keys.

Chaining: buckets and linked records

Insert 4,9,15,20,26,31 at heads; m=6, h(k)=k mod6.



Blue: buckets. Green: records. Absent14 still scans bucket2; a hash does not prove membership.

Chaining (continued)

Let m=6 and h(k)=k mod 6. Insert 4,9,15,20,26,31 in that order at chain heads. Bucket 2 is 26→20, bucket 3 is 15→9, bucket 1 contains 31, and bucket 4 contains 4. Finding 20 checks two records. An unsuccessful search for 14 also scans bucket 2's two records. This specific table is not the expected-cost formula over random h.

Head insertion takes O(1) if the key is already known to be new. Our dictionary's update semantics require search to avoid duplicates, so complete insert is not unconditionally worst-case O(1). Unlinking a valid handle in a doubly linked chain takes O(1); deletion by key still pays for search.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two expected chaining-search costs

Let $\alpha=n/m$, which may exceed 1. Simple uniform hashing assumes independent uniform slots for distinct fixed keys and $O(1)$ hash evaluation. For an absent query k and each stored x, the indicator $I_x=[h(x)=h(k)]$ has expectation $1/m$. Thus the expected scan length is $E[\Sigma I_x]=n/m=\alpha$; fixed indexing overhead gives $\Theta(1+\alpha)$.

Successful search additionally assumes that every stored record is equally likely to be queried. Fix insertion order $x_1,\ldots,x_n$, use head insertion, and perform no deletions. Searching for x_i checks itself and later keys in the same bucket, so

$$E[\text{successful record checks}] = (1/n)\Sigma_i(1+(n-i)/m) = 1+(n-1)/(2m).$$

This precise expression uses the query distribution and insertion order. It does not follow from equally likely successful queries to each bucket: a long chain contains more keys and is more likely to be selected. For only an $O(1+\alpha)$ upper bound, bound the entire chain containing the key without requiring uniform successful queries.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Two expected chaining-search costs (continued)

Constant expected cost requires $\alpha = O(1)$. A fixed table may still have a chain of length n and a worst-case $\Theta(n)$ operation. Resizing and choosing a new h require moving and rehashing records; Chapter 17 addresses amortization of dynamic-table costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Division and multiplication methods

The division method is h(k)=k mod m. If m=2^p it keeps only the lowest p bits. At m=8, 0 collides with 8 and 6 with 14. A prime modulus sometimes avoids a particular pattern, but this is no uniformity theorem for arbitrary key sets, nor does it remove the pigeonhole limitation.

Horner evaluation hashes a long string incrementally: $H_0=0$ and $H_{i+1}=(B \cdot H_i+c_i) \bmod m$. This computes the complete base-B integer modulo m without constructing a huge integer, in Θ(number of characters). If $B \equiv 1 \bmod m$, all character permutations have the same sum and collide. The formula alone guarantees no useful distribution.

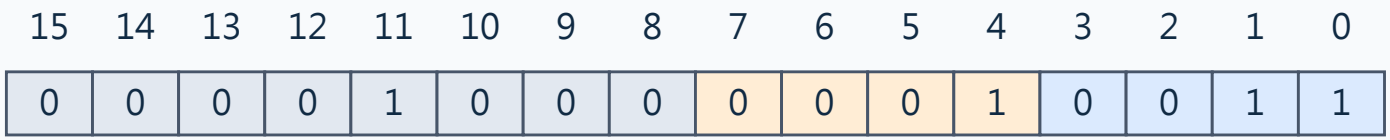
The multiplication method uses $0<A<1$ and $h(k)=\lfloor m \cdot \operatorname{frac}(kA) \rfloor$. With $A=s/2^w$ and $m=2^p$, compute the low w-bit word r_0 of $k \cdot s$, then take its highest p bits. Do not take the highest p bits of the entire 2w-bit product.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Division and multiplication methods (continued)

Multiplication hashing: high bits of the low word

w=8, s=159, k=13; product2067=8·256+19; m=16 uses p=4 bits.



High word 00001000 = 8

Low word 00010011 = 19

Orange: high four bits of low word, 0001 → h=1.

Equivalently: (2067 mod256) >>4 = 19 >>4 = 1.

Gray: discarded high word. Orange: selected bits. Blue: remaining low bits.

For w=8, s=159, k=13, the product is 2067=8・256+19. The low word is 00010011. With m=16, its high four bits are 0001, giving h=1; ⌊16・19/256⌋ also equals 1.

Universality is a probability guarantee

A family H is universal if, for every fixed pair of distinct keys x,y, uniformly choosing h from H gives $\Pr[h(x)=h(y)] \leq 1/m$. The key set and queries must be fixed independently of this choice of h. Keys need not themselves be naturally uniform.

For a fixed query k, sum collision-pair indicators. If k is absent, $E[\text{chain length}] \leq n/m$. If present, the bound is $1+(n-1)/m$. This needs only pairwise collision bounds, neither independent collision events nor uniform marginal output at every slot.

After initializing m slots, a fixed sequence of operations with at most O(m) simultaneous records gives expected O(1) per search and duplicate-checking insert, hence expected O(t) for t operations. Initialization still costs $\Theta(m)$ separately. An adversary that sees h or adaptively chooses keys from responses does not automatically satisfy the fixed-pair premise. Universality is also not cryptographic unpredictability; it does not require h to remain permanently secret.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Collision proof for a prime affine family

Choose a prime P larger than every legal key. Choose a uniformly from $1..P-1$ and b from $0..P-1$, and define $h_{a,b}(k)=((ak+b) \bmod P) \bmod m$.

For fixed $x \neq y$, set $r=(ax+b) \bmod P$ and $s=(ay+b) \bmod P$. Both a and $x-y$ are nonzero in the field F_P, so $r-s=a(x-y) \neq 0$. Conversely, each $r \neq s$ uniquely determines $a=(r-s)(x-y)^{-1} \bmod P$ and $b=r-ax \bmod P$. Uniform (a,b) therefore corresponds to the $P(P-1)$ uniformly chosen ordered distinct pairs (r,s).

For each r, at most $\lceil P/m \rceil - 1 \leq (P-1)/m$ of the other $P-1$ possible s share its residue modulo m. Collision probability is at most $1/m$, proving universality. If $m \geq P$, the final modulo operation merges no distinct r,s and collision probability is zero.

Primality makes every nonzero $x-y$ invertible. The proof does not transfer to an arbitrary composite modulus. $O(1)$ arithmetic assumes P, modular multiplication, and keys fit the specified word-RAM model; arbitrary-precision integer costs must be counted separately.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Open addressing

All records occupy the m slots themselves, so $n \leq m$. Each key has a fixed probe sequence $h(k,0),\ldots,h(k,m-1)$, reproduced by both insertion and search. To find a slot whenever one is empty, the sequence must cover every slot, usually as a permutation.

Slots have three states: EMPTY, never used since construction; DELETED, a tombstone left by deletion; or OCCUPIED(record). Search returns on an equal key, rejects at EMPTY, and continues past DELETED. It makes at most m probes. Even with no EMPTY slot, it must terminate rather than wait forever for an empty slot.

Stopping at EMPTY is correct because a key inserted farther along could not have skipped an EMPTY slot at insertion time. Later deletion changes slots to DELETED, never EMPTY, so it cannot create a new stopping gap along a surviving key's path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Tombstones and duplicate-free insertion

```
insert(k,value):
    first_deleted=NIL
    for i=0..m-1:
        j=h(k,i)
        if T[j] is OCCUPIED and T[j].key==k:
            update value; return j
        if T[j] is DELETED and first_deleted==NIL:
            first_deleted=j
        if T[j] is EMPTY:
            place at first_deleted if set, otherwise j; return
    if first_deleted!=NIL: place there; return
    error full
```

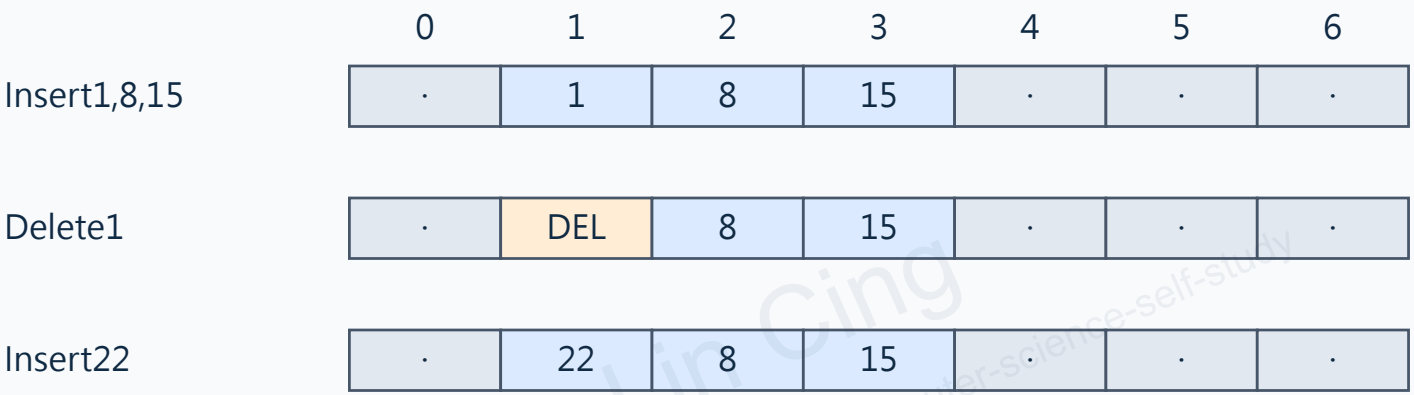
Do not insert immediately at the first DELETED slot: the same key may appear later, creating a duplicate record. The later duplicate search may be omitted only if the caller already guarantees a new key. Delete searches first, marks a found slot DELETED, and changes nothing if absent.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Tombstones and duplicate-free insertion (continued)

Tombstones preserve probe paths and unique keys

m=7, home=k mod7, linear probing. Find15 follows1→2→3 across DEL.



Insert8 updates slot2. Insert22 saves DEL1, probes to EMPTY4, then writes slot1.

Gray EMPTY: stop. Orange DELETED: continue. Blue OCCUPIED: compare key.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Preserving search on the same probe path

Let m=7 and home(k)=k mod 7. Inserting 1,8,15 places them in slots 1,2,3. Deleting 1 makes slot 1 DELETED. Searching for 15 still probes 1→2→3 and returns slot 3; it cannot stop at 1.

insert(8,new-value) remembers reusable slot 1 but continues to slot 2, finds 8, and updates it without creating another 8 at slot 1. Inserting new key 22 probes 1(DELETED), 2(8), 3(15), and 4(EMPTY). Only after confirming absence does it place 22 at the first DELETED slot, slot 1.

Lower active load α=n/m does not guarantee shorter probing. Many tombstones can leave no EMPTY slot even with few live keys, so an unsuccessful search takes m probes. The clean expected bounds below assume no deletions or tombstones. Dynamic workloads must separately limit their fraction or rebuild and account for that cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Probe formulas and coverage

Linear probing, $(h_0(k)+i) \bmod m$, always covers the table. Adjacent clusters absorb later keys whose homes lie there, producing primary clustering. Equal homes imply equal sequences.

Quadratic probing, $(h_0(k)+c_1 i+c_2 i^2) \bmod m$, does not cover the table for arbitrary parameters. With $m=7$, offsets i^2 are 0,1,4,2,2,4,1, so insertion can fail despite empty slots. Equal homes still imply equal sequences, called secondary clustering.

A full-coverage version uses $m=2^w$ and $f(i)=i(i+1)/2$, dividing as integers before reducing modulo m. If $f(i)=f(j) \bmod 2^w$, then 2^{w+1} divides $(i-j)(i+j+1)$. The factors have opposite parity, so the even factor must be divisible by 2^{w+1}. For $0 \le i,j < 2^w$, this is possible only with $i-j=0$. Thus $i=j$ and the sequence is a permutation. At $m=8$, offsets are 0,1,3,6,2,7,5,4.

Double hashing uses $(h_1(k)+i \cdot h_2(k)) \bmod m$, with period $m/\gcd(h_2(k),m)$. Write $m=gm'$ and $h_2=gd'$, with $\gcd(m',d')=1$. Returning home is equivalent to m' dividing i, so the least positive period is m'. Full coverage requires a coprime step: a prime m with nonzero step, or a power-of-two m with odd step, suffices.

Unsuccessful search under ideal uniform probing

Uniform hashing here means that each key's probe permutation is uniform over all m! permutations, and a fresh query remains an unbiased uniform permutation relative to the current table. This is stronger than uniform single-bucket hashing for chaining. Linear, quadratic, and double hashing cannot simply be assumed to satisfy it.

With no tombstones and n<m, let X count probes through the first EMPTY in an unsuccessful search. The probability that its first i−1 slots are occupied is

$\Pr(X \geq i) = \prod_{j=0}^{i-2} (n-j)/(m-j) \leq (n/m)^{i-1} = \alpha^{i-1}$ for $i \leq n+1$; later tails are zero.

For a nonnegative integer X, $X = \sum_{i \geq 1} I[X \geq i]$. Taking expectations gives

$E[X] = \sum \Pr(X \geq i) \leq \sum_{i \geq 1} \alpha^{i-1} = 1/(1-\alpha)$.

The product samples without replacement; it does not assume independent probes. Insertion of a new key through the first EMPTY has the same cost. For m=10,n=8 the bound is 5, not exactly five probes per query and not a guarantee for actual linear probing.

Successful search averages insertion loads

Fix an insertion sequence of n distinct keys with no deletion, and query a stored key uniformly. At insertion of key i+1, only i records are present, so expected probes are at most m/(m−i). Later operations neither move this key nor delete the occupied slots on its path. Search reproduces its insertion path.

```
E[successful probes] ≤ (1/n)Σ_(i=0)^(n−1) m/(m−i) = (1/α)Σ_(k=m−n+1)^m1/k

≤(1/α)∫_(m−n)^m dx/x = (1/α)ln(1/(1−α)).
```

Because 1/x decreases, its integral on [k−1,k] is at least 1/k. At α=0 there is no successful query; do not substitute zero into 1/α. The bound tends to 1 as α→0. Both expected bounds diverge as α→1, motivating spare capacity. The threshold depends on the implementation and cost model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Perfect hashing

A static dictionary's key set remains fixed after construction. The FKS two-level scheme gives worst-case constant memory accesses per query after construction, with O(n) space. Random retries occur during construction, whose time is not worst-case O(n).

For the empty set, return absent without modulo zero. Assume $n \geq 1$. The first level has $m=n$ buckets and a universal random h; bucket j has n_j keys. If Y counts unordered colliding key pairs, $\Sigma n_j^2 = n+2Y$. Each pair collides with probability at most $1/n$, so

$$E[\Sigma n_j^2] \leq n+2 \cdot \binom{n}{2}/n = 2n-1.$$

Markov gives $\Pr(\Sigma n_j^2 \geq 4n) < 1/2$. Retry until $\Sigma n_j^2 < 4n$, requiring fewer than two trials in expectation, each taking O(n) to distribute keys. Expected linear space alone does not establish a definite linear bound for an accepted table; actually check the acceptance condition.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Why square-size second levels succeed quickly

For a bucket of $s=n_j\geq 2$ keys, allocate s^2 slots and independently choose a universal h_j. Its collision-pair count Z satisfies $E[Z]\leq \binom{s}{2}/s^2=(s-1)/(2s)<1/2$. Thus $\Pr(Z\geq 1)\leq E[Z]<1/2$. Accept only after checking zero collisions; fewer than two trials are needed in expectation.

If each attempt allocates or clears s^2 slots, it costs $O(s^2)$. The accepted first level has $\Sigma s^2<4n$, so all second-level construction costs total expected $O(n)$, still $O(n)$ including the first level. Account separately for first-level retries and second-level retries conditional on the accepted buckets. An average for one bucket does not replace the total-cost argument.

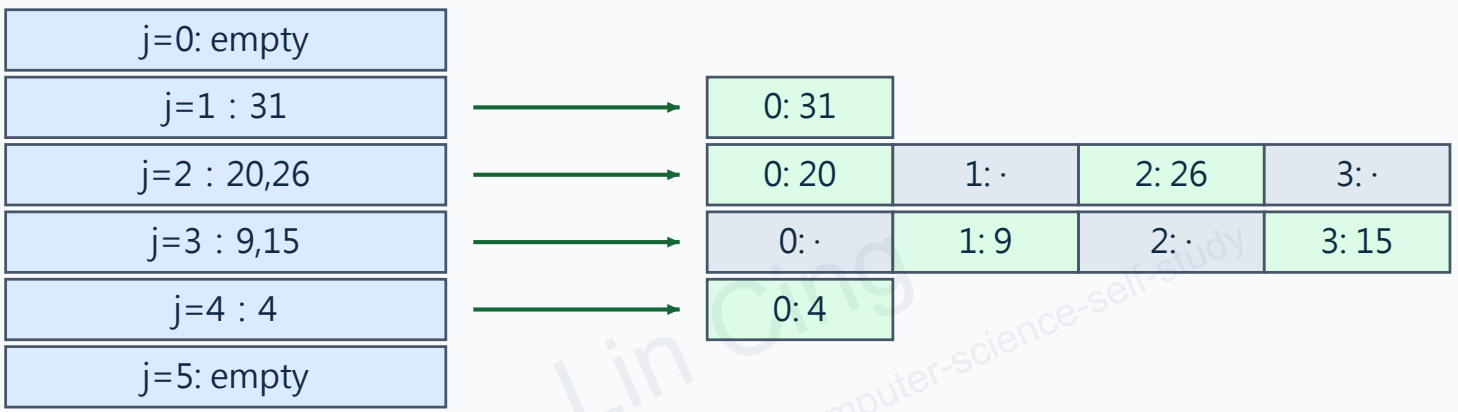
Empty buckets need no second level; singleton buckets can store the record directly, without modulo zero. Lookup computes $j=h(k)$, then $h_j(k)$ if nonempty, reads the slot, and compares the full key. An absent query may reach an occupied slot; collision freedom among stored keys does not prove membership.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Actual two-level layout and an absent query

Perfect hashing: square tables and a final key check

P=37; first h=(k mod37) mod6; two-key buckets use h_j=(k mod37) mod4.



Second levels: 1+4+4+1=10 slots. Both26 and14 reach j2→slot2; compare with26.
Blue: first level. Green: records. Gray: EMPTY. 26 succeeds; 14 fails as26≠14.

Use K={4,9,15,20,26,31}, P=37, and h(k)=(k mod 37) mod 6. Bucket sizes are [0,1,2,2,1,0], and total square-size space is 0+1+4+4+1+0=10<24. Both two-key buckets use h_j(k)=(k mod 37) mod 4, producing [20,EMPTY,26,EMPTY] and [EMPTY,9,EMPTY,15], with no collisions.

Actual two-level layout and an absent query (continued)

Query 26 follows first-level 2→second-level 2→record 26 and succeeds by equality. Query 14 follows the same 2→2 but reads 26≠14 and must reject. Each fixed query takes a constant number of word operations and memory accesses, a worst-case guarantee after successful construction. Adding keys or changing hash parameters may break collision freedom and requires rebuilding or verification.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 11.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study